시스템 통합 (System Integration)

개요

Monkey 프로그래밍 언어의 인터프리터와 컴파일러 시스템은 여러 개의 독립적인 컴포넌트들이 유기적으로 연결되어 동작합니다. 이 문서에서는 REPL(Read-Eval-Print Loop) 시스템을 중심으로 각 컴포넌트들이 어떻게 통합되어 전체 시스템을 구성하는지 상세히 분석합니다.

1. REPL 시스템과 Start() 함수 분석

1.1 REPL의 역할과 구조

REPL(Read-Eval-Print Loop)은 사용자와 Monkey 언어 시스템 간의 인터페이스 역할을 합니다. `repl/repl.go` 파일의 `Start()` 함수가 이 역할을 담당합니다.

```
func Start(in io.Reader, out io.Writer) {
scanner := bufio.NewScanner(in)
// 컴파일러 상태 초기화
constants := []object.Object{}
globals := make([]object.Object, vm.GlobalSize)
symbolTable := compiler.NewSymbolTable()
// 내장 함수들을 심볼 테이블에 등록
for i, v := range object.Builtins {
symbolTable.DefineBuiltin(i, v.Name)
}
```

```
// 메인 루프 시작
for {
// 사용자 입력 처리...
}
}
```

1.2 Start() 함수의 메인 루프 구조

Start() 함수는 다음과 같은 단계로 구성된 무한 루프를 실행합니다:

1.2.1 사용자 입력 처리

```
fmt.Printf(Prompt) // ">> " 프롬프트 출력
scanned := scanner.Scan()
if !scanned {
return  // EOF 또는 에러 시 종료
}
line := scanner.Text() // 사용자 입력 읽기
```

1.2.2 토큰화 및 파싱

```
l := lexer.New(line) // 렉서 생성
p := parser.New(l) // 파서 생성
program := p.ParseProgram() // AST 생성
```

```go
// 파싱 에러 처리
if len(p.Errors()) != 0 {
printParseErrors(out, p.Errors())
continue
}
```

1.2.3 컴파일 과정

```go
comp := compiler.NewWithState(symbolTable, constants)
err := comp.Compile(program)
if err != nil {
fmt.Fprintf(out, "Woops! Compilation fialed:\n %s\n", err)
continue
}

code := comp.Bytecode()
constants = code.Constants  // 상수 풀 업데이트
```

1.2.4 가상 머신 실행

```go
machine := vm.NewWithGlobalsStore(code, globals)
err = machine.Run()
if err != nil {
fmt.Fprintf(out, "Woops! Executing bytecode fialed:\n %s\n", err)
```

```
continue

}
```

1.2.5 결과 출력

```
lastPopped := machine.LastPoppedStackElem()

io.WriteString(out, lastPopped.Inspect())

io.WriteString(out, "\n")
```

1.3 상태 관리와 지속성

REPL은 세션 동안 다음 상태들을 유지합니다:

- **constants**: 컴파일된 상수들의 누적 저장
- **globals**: 전역 변수들의 저장소
- **symbolTable**: 심볼 정의와 스코프 관리

이를 통해 이전에 정의된 변수나 함수를 다음 입력에서도 사용할 수 있습니다.

2. 컴포넌트 간 데이터 흐름과 통합 과정

2.1 전체 데이터 흐름 개요

Monkey 언어 시스템의 데이터 흐름은 다음과 같은 단계를 거칩니다:

```
소스 코드 (string)

↓

렉서 (Lexer) → 토큰 스트림 ([]Token)
```

```
↓
파서 (Parser) → 추상 구문 트리 (AST)
↓
컴파일러 (Compiler) → 바이트코드 (Bytecode)
↓
가상 머신 (VM) → 실행 결과 (Object)
```

2.2 각 단계별 데이터 변환과 전달 방식

2.2.1 소스 코드 → 토큰 스트림

입력: 문자열 형태의 소스 코드

```
input := "let x = 5 + 10;"
```

처리: Lexer가 문자 단위로 스캔하여 토큰 생성

```
l := lexer.New(input)
tokens := []token.Token{}
for tok := l.NextToken(); tok.Type != token.EOF; tok = l.NextToken() {
tokens = append(tokens, tok)
}
```

출력: 토큰 스트림

```
[
```

```
{Type: "LET", Literal: "let"},
{Type: "IDENT", Literal: "x"},
{Type: "=", Literal: "="},
{Type: "INT", Literal: "5"},
{Type: "+", Literal: "+"},
{Type: "INT", Literal: "10"},
{Type: ";", Literal: ";"}
]
```

2.2.2 토큰 스트림 → AST

입력: 토큰 스트림

처리: Parser가 Pratt Parsing 알고리즘으로 구문 분석

```
p := parser.New(l)
program := p.ParseProgram()
```

출력: AST 노드 구조

```
&ast.Program{
Statements: []ast.Statement{
&ast.LetStatement{
Token: {Type: "LET", Literal: "let"},
Name: &ast.Identifier{Value: "x"},
Value: &ast.InfixExpression{
```

```
Left: &ast.IntegerLiteral{Value: 5},
Operator: "+",
Right: &ast.IntegerLiteral{Value: 10}
}
}
}
}
```

2.2.3 AST → 바이트코드

입력: AST 노드 구조

처리: Compiler가 AST를 순회하며 바이트코드 생성

```
comp := compiler.New()
err := comp.Compile(program)
bytecode := comp.Bytecode()
```

출력: 바이트코드와 상수 풀

```
&compiler.Bytecode{
Instructions: []byte{
code.OpConstant, 0, 0, // 상수 0 (값: 5) 로드
code.OpConstant, 0, 1, // 상수 1 (값: 10) 로드
code.OpAdd, // 덧셈 연산
code.OpSetGlobal, 0, 0, // 전역 변수 0에 저장
```

```
code.OpPop  // 스택에서 제거
},
Constants: []object.Object{
&object.Integer{Value: 5},
&object.Integer{Value: 10}
}
}
```

2.2.4 바이트코드 → 실행 결과

입력: 바이트코드와 상수 풀

처리: VM이 스택 기반으로 명령어 실행

```
machine := vm.New(bytecode)
err := machine.Run()
result := machine.LastPoppedStackElem()
```

출력: 객체 형태의 실행 결과

```
&object.Integer{Value: 15}
```

2.3 컴포넌트 간 인터페이스

2.3.1 Lexer ↔ Parser 인터페이스

```
// Parser가 Lexer로부터 토큰을 요청
```

```go
type Parser struct {
l *lexer.Lexer
currentToken token.Token
peekToken token.Token
}

func (p *Parser) nextToken() {
p.currentToken = p.peekToken
p.peekToken = p.l.NextToken()
}
```

2.3.2 Parser ↔ Compiler 인터페이스

```go
// Compiler가 AST 노드를 처리
func (c *Compiler) Compile(node ast.Node) error {
switch node := node.(type) {
case *ast.Program:
for _, s := range node.Statements {
err := c.Compile(s)
if err != nil {
return err
}
}
// ... 다른 노드 타입들 처리
```

```
}

}
```

2.3.3 Compiler ↔ VM 인터페이스

```
// VM이 컴파일된 바이트코드를 실행

func  New(bytecode *compiler.Bytecode) *VM {

return &VM{

constants: bytecode.Constants,

// 바이트코드의 Instructions를 실행 가능한 형태로 변환

}

}
```

2.4 에러 전파와 처리

각 컴포넌트에서 발생한 에러는 상위 레벨로 전파됩니다:

1. **Lexer 에러**: 잘못된 문자 → `ILLEGAL` 토큰 생성
2. **Parser 에러**: 구문 오류 → `p.Errors()` 슬라이스에 수집
3. **Compiler 에러**: 컴파일 오류 → `error` 반환
4. **VM 에러**: 런타임 오류 → `error` 반환

REPL에서는 각 단계의 에러를 적절히 처리하여 사용자에게 표시합니다.## 3. 인터프리터와 컴파일러 성능 비교

3.1 두 실행 방식의 차이점과 장단점

Monkey 언어는 두 가지 실행 방식을 지원합니다: 인터프리터 방식과 컴파일러+VM 방식입니다.

3.1.1 인터프리터 방식 (Evaluator)

동작 원리:

- AST를 직접 순회하며 노드별로 즉시 평가
- 각 노드 타입에 대해 `Eval()` 함수가 재귀적으로 호출
- 실행 시점에 타입 검사와 연산 수행

```
func Eval(node ast.Node, env *object.Environment) object.Object {
switch node := node.(type) {
case *ast.Program:
return evalProgram(node, env)
case *ast.InfixExpression:
left := Eval(node.Left, env)
right := Eval(node.Right, env)
return evalInfixExpression(node.Operator, left, right)
// ... 다른 노드 타입들
}
}
```

장점:

- 구현이 단순하고 직관적
- 디버깅이 용이 (AST와 실행이 1:1 대응)
- 메모리 사용량이 적음

- 동적 기능 구현이 쉬움

단점:

- 실행 속도가 느림 (매번 AST 순회)
- 반복문에서 성능 저하가 심함
- 타입 검사를 매번 수행

3.1.2 컴파일러+VM 방식

동작 원리:

- AST를 바이트코드로 미리 컴파일
- VM이 스택 기반으로 바이트코드 실행
- 최적화된 명령어 세트 사용

```
// 컴파일 단계
comp := compiler.New()
comp.Compile(program)
bytecode := comp.Bytecode()


// 실행 단계
machine := vm.New(bytecode)
machine.Run()
```

장점:

- 실행 속도가 빠름 (바이트코드 최적화)
- 반복 실행 시 성능 우수

- 메모리 접근 패턴이 효율적
- 추가 최적화 가능

단점:

- 구현 복잡도가 높음
- 컴파일 오버헤드 존재
- 메모리 사용량이 많음 (바이트코드 + 스택)
- 디버깅이 어려움

3.2 메모리 사용량과 실행 속도 비교

3.2.1 벤치마크 테스트 분석

프로젝트의 `benchmark/main.go` 는 피보나치 수열 계산을 통해 두 방식의 성능을 비교합니다:

```
// 테스트 코드: fibonacci(35) 계산
let  fibonacci = fn(x) {
if (x == 0) {
0
} else {
if (x == 1) {
return  1;
} else {
fibonacci(x - 1) + fibonacci(x - 2);
}
}
};
```

```
fibonacci(35);
```

실행 방법:

```
# 인터프리터 방식
go run benchmark/main.go -engine=eval

# 컴파일러+VM 방식
go run benchmark/main.go -engine=vm
```

3.2.2 성능 특성 분석

실행 속도:

- **컴파일러+VM**: 일반적으로 3-10배 빠름
- **인터프리터**: 단순한 연산에서는 차이가 적음
- **반복/재귀**: VM이 압도적으로 우수

메모리 사용량:

- **인터프리터**:
- AST 노드 + Environment 객체
- 재귀 호출 스택 (함수 호출 시)
- 약 50-100MB (복잡한 프로그램)
- **컴파일러+VM**:
- 바이트코드 + 상수 풀 + VM 스택
- 고정 크기 스택 (2048 슬롯)

- 약 100-200MB (바이트코드 포함)

시작 시간:

- **인터프리터**: 즉시 실행 가능
- **컴파일러+VM**: 컴파일 시간 추가 (보통 수 ms)

3.3 적합한 사용 사례 분석

3.3.1 인터프리터 방식이 적합한 경우

1. **개발 및 디버깅 단계**

- 빠른 프로토타이핑
- 대화형 개발 환경
- 에러 추적이 중요한 경우

2. **단순한 스크립트**

- 일회성 실행
- 짧은 코드
- 설정 파일 처리

3. **교육 목적**

- 언어 학습
- 알고리즘 실험
- 개념 이해

3.3.2 컴파일러+VM 방식이 적합한 경우

1. **성능이 중요한 애플리케이션**

- 수치 계산
- 반복 처리가 많은 작업

- 실시간 시스템

2. **프로덕션 환경**

- 서버 애플리케이션
- 배치 처리
- 장시간 실행되는 프로그램

3. **최적화가 필요한 경우**

- 메모리 효율성
- CPU 집약적 작업
- 대용량 데이터 처리

3.4 성능 최적화 전략

3.4.1 컴파일러 최적화

- 상수 폴딩 (Constant Folding)
- 데드 코드 제거 (Dead Code Elimination)
- 인라인 함수 최적화

3.4.2 VM 최적화

- 명령어 융합 (Instruction Fusion)
- 스택 최적화
- 가비지 컬렉션 최적화

3.4.3 하이브리드 접근법

- JIT 컴파일 (Just-In-Time)
- 적응형 최적화 (Adaptive Optimization)

- 프로파일 기반 최적화 (Profile-Guided Optimization)

결론

Monkey 언어의 시스템 통합은 각 컴포넌트가 명확한 인터페이스를 통해 연결되어 있어 유지보수성과 확장성을 제공합니다. REPL을 통한 대화형 환경과 두 가지 실행 방식의 선택권은 다양한 사용 사례에 대응할 수 있는 유연성을 제공합니다. 성능과 개발 편의성 사이의 트레이드오프를 이해하고 적절한 실행 방식을 선택하는 것이 중요합니다.